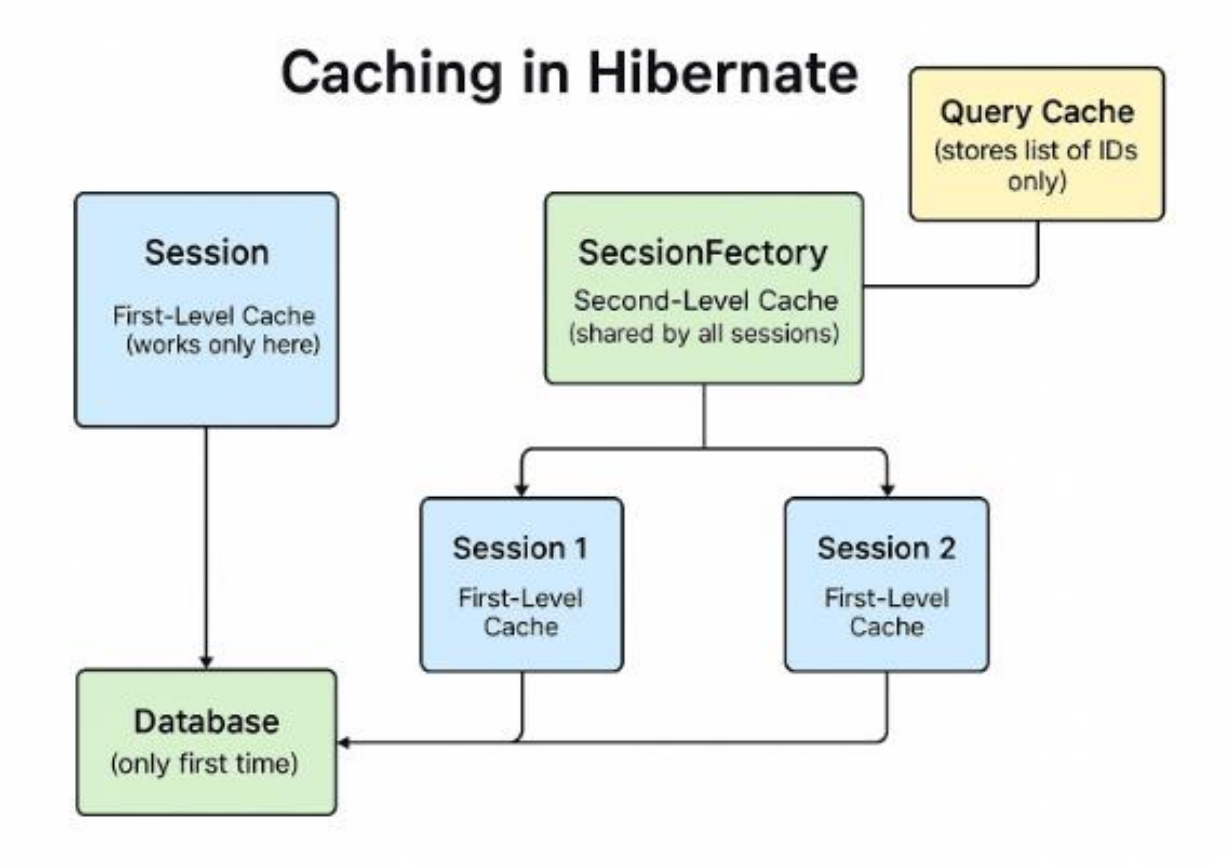
Caching in Hibernate

Caching in Hibernate improves performance by reducing the number of database queries. Instead of hitting the database every time, Hibernate stores frequently accessed data in memory so it can be reused.

Hibernate provides **two main levels of caching**:



---

1. First-Level Cache (Session Cache)

Enabled by default — you cannot disable it.
It is **associated with the Hibernate Session object**.
Works **per request**, meaning each session has its own cache.

Data stored here lasts **only until the session is closed**.
Prevents repeated queries for the same entity within the same session.

Example:

```
Session session = sessionFactory.openSession();
Student s1 = session.get(Student.class, 1); // DB hit
Student s2 = session.get(Student.class, 1); // No DB hit (from cache)
```

---

2. Second-Level Cache (SessionFactory Cache)

Optional — must be explicitly enabled.
Shared across **multiple sessions**.
Works with the **SessionFactory**, so cached data is available to all sessions.
Stores entities, collections, or query results depending on configuration.
Helps reduce database load across the entire application.

Common Second-Level Cache Providers:

- **EHCache**
- **Infinispan**
- **OSCache**
- **Hazelcast**

---

3. Query Cache (Optional – Works with Second-Level Cache)

Stores **query results**, not the entities themselves.
Requires **second-level cache** to be enabled.
Mainly used for caching the list of IDs returned by queries.

Example:

```
Query q = session.createQuery("from Student");
q.setCacheable(true); // Enables query cache
```

Summary Table

Cache Level	Default?	Scope	Stores	Purpose
1st Level	Yes	Per Session	Entities	Avoid repeated DB calls within the session
2nd Level	No	SessionFactory-wide	Entities, Collections	Reduce DB load across sessions
Query Cache	No	SessionFactory-wide	Query results	Speed up repeated queries